PRÁCTICA OPENCODE UT3. CONTENERIZAR Y AUDITAR CON UN AGENTE

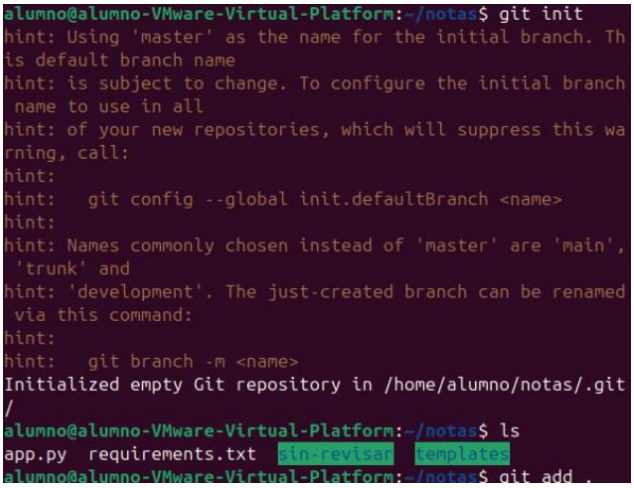
Antes de empezar

opencode instalado (ver la **guía de instalación**) y el proyecto en un repositorio Git, que es tu botón de deshacer. Comprueba con `/models` qué modelo tienes activo: con los modelos gratuitos tendrás que ser más explícito en los prompts y cometerán más errores, lo que viene bien para lo que se pide aquí. Necesitas además Docker Engine y el plugin Compose v2 (`docker --version` y `docker compose version`) y la aplicación `notas` de la carpeta `recursos`. Si el puerto 8000 está ocupado en tu equipo, cambia el puerto publicado y dilo en la entrega.

- Trabaja en **modo Plan** (tecla Tab) hasta que el plan te convenza; solo entonces pasa a **Build**.
- Revisa cada cambio antes de aceptarlo (`git diff`). Si no sabes explicar una línea, no la aceptes.
- **Verifica ejecutando**: la evidencia es la salida del comando, no la respuesta del agente.
- Anota cada prompt en `prompts.md`: qué pediste, qué hizo y qué corregiste.
- Nada de contraseñas ni datos personales en el chat: los modelos gratuitos son de prueba y no admiten información confidencial.
- Si el modelo se atasca, da vueltas o va muy lento, divide la petición en pasos o cambia de modelo con `/models`.

Ejercicios

1) **Análisis previo.** Copia `notas` a tu carpeta de trabajo, inicialízala como repositorio Git y arranca opencode. En **modo Plan**, pídele que explique qué hace la aplicación, qué necesita para arrancar (puerto, variables de entorno, dependencias) y qué estrategia de contenerización propone. Comprueba en `app.py` y `requirements.txt` que lo que dice es cierto. (0,5p)



```
alumno@alumno-VMware-Virtual-Platform:~/notas$ git commit -
m "Estado Inicial"
[master (root-commit) f5ba17f] Estado Inicial
 6 files changed, 96 insertions(+)
 create mode 100644 app.py
 create mode 100644 requirements.txt
 create mode 100644 sin-revisar/Dockerfile
 create mode 100644 sin-revisar/LEEME.txt
 create mode 100644 sin-revisar/docker-compose.yml
 create mode 100644 templates/index.html
```

El agente dijo que sin Redis la app no rompe, pero es falso, porque solo `/salud` captura `RedisError` y `/` y `POST /notas` devuelven un 500. Propuso un healthcheck con `curl`, que `python:3.12-slim` no trae, así que hay que usar Python con `urllib`. El `.dockerignore` no incluía `.env`, y presentó como opcional la caché de `requirements.txt`, que en el enunciado es obligatoria. No indicó que `REDIS_URL` debe ser `redis://cache:6379` en el compose, ni añadió `--appendonly yes` para la persistencia de Redis. Además se adelantó a auditar `sin-revisar/`, que es el ejercicio 5 y lo tienes que verificar tú.

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# cat app.py
import os

import redis
from flask import Flask, redirect, render_template, request

app = Flask(__name__)
cache = redis.from_url(os.environ.get("REDIS_URL", "redis://localhost:6379"),
                       decode_responses=True)

TITULO = os.environ.get("TITULO_APP", "Tablón de notas")


@app.get("/")
def portada():
    notas = cache.lrange("notas", 0, 49)
    return render_template("index.html", titulo=TITULO, notas=notas)


@app.post("/notas")
def crear():
    texto = (request.form.get("texto") or "").strip()
    if texto:
        cache.lpush("notas", texto)
        cache.ltrim("notas", 0, 49)
    return redirect("/")


@app.get("/salud")
def salud():
    try:
        cache.ping()
    except redis.RedisError:
        return {"estado": "sin cache"}, 503
    return {"estado": "ok", "notas": cache.llen("notas")}


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, debug=True)
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# cat requirements.txt
Flask==3.0.3
redis==5.0.4
gunicorn==22.0.0
```

2) **Dockerfile y .dockerignore.** Pídele el `Dockerfile` exigiendo por escrito: imagen base `python:3.12-slim`, dependencias instaladas **antes** de copiar el código para aprovechar la caché, ejecución con un **usuario sin privilegios**, `EXPOSE 8000`, `HEALTHCHECK` contra `/salud`, arranque con **gunicorn** (no con el servidor de desarrollo de Flask) y `CMD` en formato JSON. Pídele también un `.dockerignore`. Revisa el resultado línea a línea y corrige lo que no cumpla. (2p)

```
Comprueba si el Dockerfile y el .dockerignore cumplen estos requisitos y dime qué línea cumple cada uno: imagen
base python:3.12-slim; dependencias instaladas antes de copiar el código; usuario sin privilegios; EXPOSE 8000;
HEALTHCHECK contra /salud; arranque con gunicorn; CMD en formato JSON; .dockerignore presente. No modifiques
nada si todo cumple.

+ Thought: 286ms

→ Read Dockerfile
→ Read .dockerignore

+ Thought: 603ms

Todo cumple (Dockerfile 19 líneas, .dockerignore 13):

| Requisito                          | Línea                                                
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker build -t notas-web .
[+] Building 4.6s (11/11) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 567B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                           1.0s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 163B                                                                                             0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:cb33f7f771ecec2ef1225cf39b159591a040391bd73e4c01f00fdfc4             0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:cb33f7f771ecec2ef1225cf39b159591a040391bd73e4c01f00fdfc4             0.1s
 => [internal] load build context                                                                                             0.0s
 => => transferring context: 132B                                                                                             0.0s
 => CACHED [2/6] WORKDIR /app                                                                                                 0.0s
 => CACHED [3/6] COPY requirements.txt .                                                                                      0.0s
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                           0.0s
 => [5/6] COPY . .                                                                                                            0.1s
 => [6/6] RUN useradd --create-home --shell /usr/sbin/nologin appuser &&     chown -R appuser:appuser /app                    1.3s
 => exporting to image                                                                                                        1.3s
 => => exporting layers                                                                                                       0.6s
 => => exporting manifest sha256:54da9ef0fd870bdd122357a1814b029e0b847cb45bb7fc051ec5f48144a59a04                             0.1s
 => => exporting config sha256:d4483d495dc9e79727824a0e0a3677ea4315a61ed40651187e439adcae6b844e                               0.0s
 => => exporting attestation manifest sha256:da61d2b4151e8bc7aaba8af157e505203b45b20ff5dd5ecbbae0c1ad6fa5a0ce                 0.1s
 => => exporting manifest list sha256:6d9ab82de00d6a81d58194bd974e8d357b134b395fae8f90d2e4d4861e5935a2                        0.0s
 => => naming to docker.io/library/notas-web:latest                                                                           0.0s
 => => unpacking to docker.io/library/notas-web:latest                                                                        0.2s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker image ls
                                                                                                  i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   11a3eb1597e8       25.9kB         9.49kB    U
notas-web:latest     6d9ab82de00d        204MB         49.5MB
redis:7-alpine       57e4255cc971       57.8MB         16.7MB    U
```

3) **Compose.** Pídele `docker-compose.yml` con dos servicios: `web` (construido del Dockerfile, publicando 8000) y `cache` (Redis con persistencia). Requisitos: Redis **no** debe publicar ningún puerto al exterior, debe tener un **volumen con nombre**, `healthcheck` y `web` debe esperar a que esté sano (`condition: service_healthy`). Las variables van en el compose, nunca dentro de la imagen. (2p)

> Comprueba si docker-compose.yml cumple estos requisitos y dime qué linea cumple cada uno: dos servicios web y cache; web construido del Dockerfile y publicando 8000; cache con Redis y persistencia; Redis sin publicar ningún puerto al exterior; volumen con nombre; healthcheck en cache; web espera con condition: service_healthy; variables definidas en el compose, no en la imagen. No modifiques nada si todo cumple.

+ Thought: 138ms

→ Read docker-compose.yml

+ Thought: 2.8s

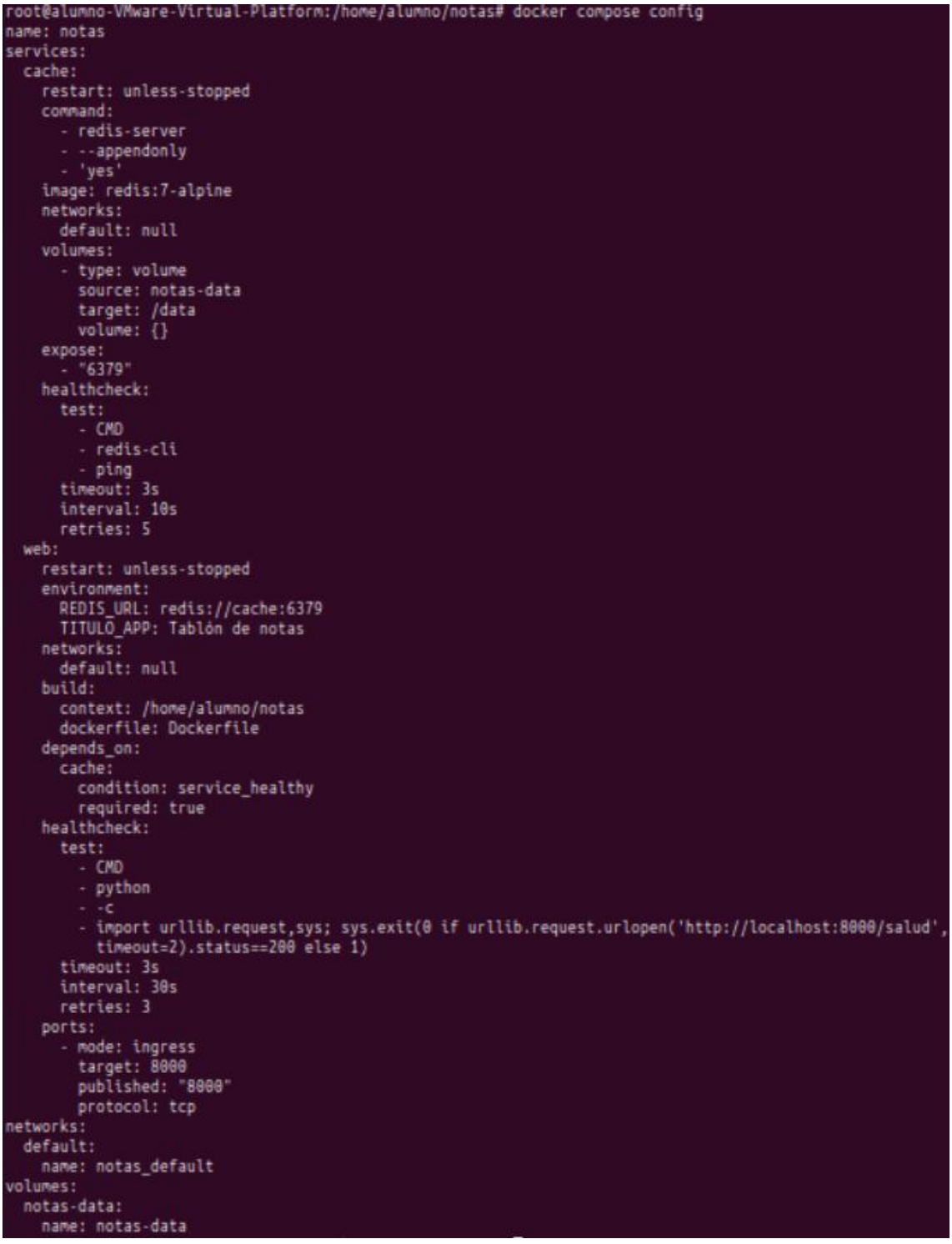
Todo cumple (docker-compose.yml, 35 lineas):

Requisito	Linea(s)
Dos servicios web y cache	docker-compose.yml:2 (web) y docker-compose.yml:19 (cache)
web construido del Dockerfile	docker-compose.yml:3 (build: .)
web publicando 8000	docker-compose.yml:4-5 (ports: "8000:8000")
cache con Redis	docker-compose.yml:20 (image: redis:7-alpine)
cache con persistencia	docker-compose.yml:21 (--appendonly yes) + docker-compose.yml:22-23 (volumen /data)
Redis sin publicar puertos al exterior	docker-compose.yml:24-25 usa expose: "6379" (solo red interna); no hay bloque ports: en cache → sin binding en el host
Volumen con nombre	docker-compose.yml:33-35 (name: notas-data)
Healthcheck en cache	docker-compose.yml:26-30 (redis-cli ping)
web espera con condition: service_healthy	docker-compose.yml:9-11
Variables en el compose, no en la imagen	docker-compose.yml:6-8 (REDIS_URL, TITULO_APP); el Dockerfile no contiene ninguna instrucción ENV

No se modifica nada.

▣ Plan · MiMo-V2.6-Flash Free · 11.6s

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# cat -n docker-compose.yml
     1  services:
     2    web:
     3      build: .
     4      ports:
     5        - "8000:8000"
     6      environment:
     7        - REDIS_URL=redis://cache:6379
     8        - TITULO_APP=Tablón de notas
     9      depends_on:
    10        cache:
    11          condition: service_healthy
    12      restart: unless-stopped
    13      healthcheck:
    14        test: ["CMD", "python", "-c", "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/salud', timeout=2)
.status==200 else 1)"]
    15        interval: 30s
    16        timeout: 3s
    17        retries: 3
    18
    19    cache:
    20      image: redis:7-alpine
    21      command: ["redis-server", "--appendonly", "yes"]
    22      volumes:
    23        - notas-data:/data
    24      expose:
    25        - "6379"
    26      healthcheck:
    27        test: ["CMD", "redis-cli", "ping"]
    28        interval: 10s
    29        timeout: 3s
    30        retries: 5
    31      restart: unless-stopped
    32
    33  volumes:
    34    notas-data:
    35      name: notas-data
```



```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose config
name: notas
services:
  cache:
    restart: unless-stopped
    command:
      - redis-server
      - --appendonly
      - 'yes'
    image: redis:7-alpine
    networks:
      default: null
    volumes:
      - type: volume
        source: notas-data
        target: /data
        volume: {}
    expose:
      - "6379"
    healthcheck:
      test:
        - CMD
        - redis-cli
        - ping
      timeout: 3s
      interval: 10s
      retries: 5
  web:
    restart: unless-stopped
    environment:
      REDIS_URL: redis://cache:6379
      TITULO_APP: Tablón de notas
    networks:
      default: null
    build:
      context: /home/alumno/notas
      dockerfile: Dockerfile
    depends_on:
      cache:
        condition: service_healthy
        required: true
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/salud',
          timeout=2).status==200 else 1)
      timeout: 3s
      interval: 30s
      retries: 3
    ports:
      - mode: ingress
        target: 8000
        published: "8000"
        protocol: tcp
networks:
  default:
    name: notas_default
volumes:
  notas-data:
    name: notas-data
```

4) **Puesta en marcha y pruebas.** Levanta la aplicación, comprueba el estado de los servicios y prueba: la portada, el alta de una nota y `/salud`. Verifica la **persistencia**: para los contenedores, vuélvelos a levantar y comprueba que las notas siguen ahí; después borra los volúmenes y comprueba que desaparecen. Si algo falla, pasa el error al agente (puedes ejecutar la orden desde el chat con `!` y adjuntar el fichero con `@`) e itera. (1,5p)

```
docker compose up -d --build
docker compose ps
curl -s localhost:8000/salud
docker compose restart
docker compose down -v
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose up -d --build
[+] Building 18.3s (13/13) FINISHED
 => [internal] load local bake definitions                          0.0s
 => => reading from stdin 472B                                      0.0s
 => [internal] load build definition from Dockerfile                0.1s
 => => transferring dockerfile: 567B                                0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim  1.4s
 => [internal] load .dockerignore                                   0.0s
 => => transferring context: 163B                                   0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:a0f57064cb  0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:a0f57064cb  0.1s
 => [internal] load build context                                   0.1s
 => => transferring context: 132B                                   0.1s
 => [2/6] WORKDIR /app                                              0.1s
 => [3/6] COPY requirements.txt .                                   0.1s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt        8.2s
 => [5/6] COPY . .                                                  0.3s
 => [6/6] RUN useradd --create-home --shell /usr/sbin/nologin appus  1.6s
 => exporting to image                                              5.1s
 => => exporting layers                                             2.6s
 => => exporting manifest sha256:d92d1bd416e059889e45c679ccb6ad089a  0.0s
 => => exporting config sha256:67ae94fdad1283c95db3fa13bb6e2a7d73a2  0.0s
 => => exporting attestation manifest sha256:0e8b99377edf0666fa9535  0.0s
 => => exporting manifest list sha256:b4867db1875c4178d68925c0d852e  0.0s
 => => naming to docker.io/library/notas-web:latest                 0.0s
 => => unpacking to docker.io/library/notas-web:latest              2.1s
 => resolving provenance for metadata file                          0.1s
[+] up 3/3
 ✔ Image notas-web          Built                                  18.8s
 ✔ Container notas-cache-1  Healthy                                 4.6s
 ✔ Container notas-web-1    Started                                 5.7s
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose ps
NAME            IMAGE           COMMAND                  SERVICE   CREATED             STATUS                       PORTS
notas-cache-1   redis:7-alpine  "docker-entrypoint.s…"   cache     13 hours ago        Up 13 hours (healthy)        6379/tcp
notas-web-1     notas-web       "gunicorn -b 0.0.0.0…"   web       About a minute ago  Up About a minute (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -s localhost:8000/salud
{"estado":"ok","notas":1}
```




```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -s localhost:8000/salud
{"estado":"ok","notas":2}
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose restart
[+] restart 0/2
 ⠸ Container notas-web-1   Restarting                                  4.9s
 ⠸ Container notas-cache-1 Restarting                                  4.9s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -s localhost:8000/salud
{"estado":"ok","notas":2}
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose down
[+] down 3/3
 ✔ Container notas-web-1   Removed                                     0.8s
 ✔ Container notas-cache-1 Removed                                     0.7s
 ✔ Network notas_default   Removed                                     0.3s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose up -d
[+] up 3/3
 ✔ Network notas_default   Created                                     0.3s
 ✔ Container notas-cache-1 Healthy                                    12.5s
 ✔ Container notas-web-1   Started                                    13.4s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -s localhost:8000/salud
{"estado":"ok","notas":2}
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose down -v
[+] down 4/4
 ✔ Container notas-web-1   Removed                                     1.1s
 ✔ Container notas-cache-1 Removed                                     0.5s
 ✔ Network notas_default   Removed                                     0.3s
 ✔ Volume notas-data       Removed                                     0.0s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose up -d
[+] up 4/4
 ✔ Network notas_default   Created                                     0.4s
 ✔ Volume notas-data       Created                                     0.0s
 ✔ Container notas-cache-1 Healthy                                    12.1s
 ✔ Container notas-web-1   Started                                    12.6s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -s localhost:8000/salud
{"estado":"ok","notas":0}
```

No hay ningún error.

5) **Auditoría de `sin-revisar` (2p).** Pide al agente un informe de problemas del `Dockerfile` y del `docker-compose.yml` de esa carpeta. Después **verifica tú cada punto** con comandos y rellena una tabla: problema señalado, ¿es cierto? (evidencia), gravedad y corrección. Debes incluir obligatoriamente:
   a. el tamaño de las dos imágenes (`docker build -f sin-revisar/Dockerfile -t notas-malo .` y `docker image ls`);
   b. qué usuario ejecuta cada contenedor;
   c. dónde acaba la variable `CLAVE_ADMIN` y cómo se puede leer desde fuera;
   d. qué servidor web arranca cada versión;
   e. al menos **un punto del informe del agente que sea falso, irrelevante o exagerado**, explicando por qué.

Audita @sin-revisar/Dockerfile y @sin-revisar/docker-compose.yml y haz un informe de problemas de seguridad, tamaño y buenas prácticas. Indica la gravedad de cada uno. No modifiques nada.

+ Thought: 988ms

→ Read sin-revisar/Dockerfile
→ Read sin-revisar/docker-compose.yml

+ Thought: 35.9s

```
$ docker images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | sort

hello-world:latest 25.9kB
notas-web:latest 204MB
redis:7-alpine 57.8MB
```

```
$ git log --oneline --all -- sin-revisar/ && git ls-files sin-revisar/

f5ba17f Estado Inicial
sin-revisar/Dockerfile
sin-revisar/LEEME.txt
sin-revisar/docker-compose.yml
```

+ Thought: 2.2s

Informe de auditoría — sin-revisar/ (solo lectura, nada modificado)

Escala: Crítica (explotable / pérdida de datos grave) · Alta · Media · Baja

A. sin-revisar/Dockerfile (13 líneas)

#	Línea	Problema	Gravedad
1	Dockerfile:11	`ENV CLAVE_ADMIN=admin1234` — **credencial en claro horneada en la imagen.** Cualquiera con la imagen la lee con `docker inspect`/`docker history` (y persiste en todas las capas y registros aunque luego se borre: exige rebuild). Además queda en el historial de git. Password débil y además la app **ni usa** esa variable.	**Crítica**
2	Dockerfile:13	`CMD python app.py` **arranca Flask con** `debug=True` (app.py:38) → consola Werkzeug expuesta = ejecución remota de código si el puerto es accesible; además es el servidor de desarrollo (1 proceso, sin WSGI). `gunicorn` está en `requirements.txt` pero nadie lo invoca.	**Crítica**
3	Dockerfile:13	Shell-form de `CMD` (`CMD python app.py`) en vez de exec/JSON: `python` no es PID 1 → no recibe SIGTERM/SIGINT, cierre brusco con `docker stop` y `restart: always` en bucle.	Media
4	Dockerfile (global)	**No hay `USER`: todo ejecuta como root dentro del contenedor** → escalada de privilegios si hay fuga en la app.	**Alta**
5	Dockerfile:1	`FROM python:latest` **sin fijar versión/digest:** builds no reproducibles, cambios de versión de Python y de paquetes base sin control (riesgo de supply-chain y de roturas).	**Alta**
6	Dockerfile:6	`COPY . .` **sin** `.dockerignore` y con build-context = raíz del repo (según `LEEME.txt:7`): arrastra `.git/`, `sin-revisar/`, caches, `venv/` y cualquier `.env` al interior de la imagen → más tamaño + fuga de historial/secretos.	**Alta**
7	Dockerfile:10	**Config horneada en la imagen** (`ENV REDIS_URL=...`): en vez de inyectarse en compose/orquestador; obliga a reconstruir para cambiar entorno y funciona solo si el host se llama `cache`.	Media
8	Dockerfile:3	`apt-get install -y python3-pip curl vim git`: `vim`, `git` **y** `curl` **no hacen falta en runtime** (herramientas de usuario → superficie de ataque y ~100 MB); `python3-pip` ya viene en la imagen base.	Media

9	Dockerfile:3	Sin rm -rf /var/lib/apt/lists/* ni --no-install-recommends → capa con listas de apt retenidas (infla tamaño y expone metadatos del build).	Media
10	Dockerfile:6-8	**Orden de capas invertido**: COPY . . antes de pip install → cualquier cambio de código invalida la capa de dependencias (reinstala todo en cada build, builds lentos).	Media
11	Dockerfile:8	pip install sin --no-cache-dir → caché de pip (≈40-80 MB) conservada en la capa.	Media
12	Dockerfile (global)	Sin HEALTHCHECK → el orquestador no sabe si la app está viva (solo depends_on sin condición la distingue).	Baja
13	Dockerfile (global)	Sin EXPOSE 8000 (documental; el publish lo hace compose).	Baja

B. sin-revisar/docker-compose.yml (15 líneas)

#	Línea	Problema	Gravedad
14	docker-compose.yml:14-15	**Redis publicado en el host** (6379:6379, **todas las interfaces**) **sin** requirepass. Un Redis expuesto = robo/borrado de datos y RCE clásica (escritura en cron/ authorized_keys, módulos/Lua). En VPS con puertos abiertos es compromiso directo.	**Crítica**
15	docker-compose.yml:7	CLAVE_ADMIN=admin1234 **secretro en claro en el compose committeado a git** (y repetido del Dockerfile). Igual que #1 pero en repo.	**Alta**
16	docker-compose.yml:13	image: redis:latest sin fijar tag → upgrades mayores inesperados, builds/ imágenes no reproducibles.	Media
17	cache (sin sección)	**Sin volumen ni** appendonly: **cero persistencia** → al recrear el servicio (docker compose up --force-recreate, cambio de imagen) se pierden **todas las notas**.	Media
18	docker-compose.yml:8-9	depends_on simple (sin condition: service_healthy) → web puede arrancar antes de que Redis acepte conexiones; sin healthcheck en ningún servicio la diferencia es invisible.	Media
19	docker-compose.yml (global)	**Sin** healthcheck **en** web **ni en** cache → nada verifica /salud ni redis-cli ping; incidencias detectadas solo cuando el usuario ve error.	Media
20	docker-compose.yml:6-7	Variables solo en web y una de ellas innecesaria; falta TITULO_APP (usa default, menor). La config efectiva (REDIS_URL) viene **de la imagen** (#7), no del compose → mezcla de responsabilidades.	Baja
21	docker-compose.yml:10	restart: always reinicia también tras parada manual del daemon/usuario; unless-stopped es el hábito habitual.	Baja
22	docker-compose.yml (global)	Sin hardening: sin cap_drop, read_only, security_opt: no-new-privileges, límites de CPU/memoria ni red interna explícita.	Baja

. Tamaño

Aspecto	Evaluación
Base python:latest (≈350 MB) + vim git curl + python3-pip + listas apt + caché pip + .git y repo completos por COPY . .	estimación **≈550-700 MB** (no medido: no construí la imagen en esta fase)
Referencia medida: la versión de la raíz (python:3.12-slim, orden de capas correcto, --no-cache-dir, sin herramientas extra)	**204 MB** (docker images)
Ahorro estimado	**~2,5-3× menor** la versión de la raiz, además de reproducible (tag fijado) y sin secretos embebidos

C. Tamaño

Aspecto	Evaluación
Base `python:latest` (≈350 MB) + `vim git curl` + `python3-pip` + listas apt + caché pip + `.git` y repo completos por `COPY . .`	estimación **≈550-700 MB** (no medido: no construí la imagen en esta fase)
Referencia medida: la versión de la raíz (`python:3.12-slim`, orden de capas correcto, `--no-cache-dir`, sin herramientas extra)	**204 MB** (`docker images`)
Ahorro estimado	**~2,5-3× menor** la versión de la raíz, además de reproducible (tag fijado) y sin secretos embebidos

(Si quieres, al salir de plan mode la construyo y mido con `docker build -f sin-revisar/Dockerfile -t notas-malo .`, tal como indica `LEEME.txt:7`.)

Resumen por gravedad

- **Críticas (3):** #1 credencial en imagen, #2 servidor de debug (RCE), #14 Redis publicado sin auth.
- **Altas (4):** #4 root, #5 tag `latest`, #6 `COPY . .` sin ignore, #15 secretos en git.
- **Medias (9):** #3, #7, #8, #9, #10, #11, #16, #17, #18, #19.
- **Bajas (5):** #12, #13, #20, #21, #22.

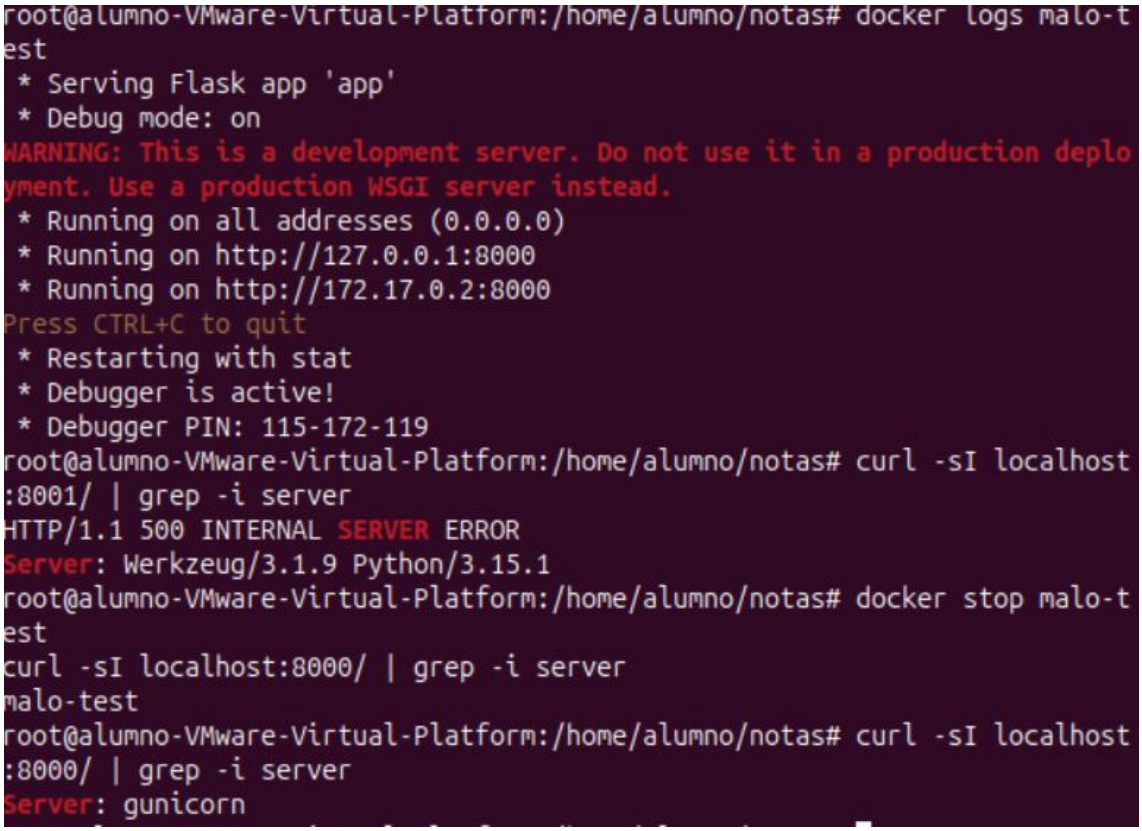
Todo lo anterior se resuelve con la implementación ya presente en la raíz (`Dockerfile`, `.dockerignore`, `docker-compose.yml`) salvo el tema de secretos, que además debería **rotarse/borrarse del historial de git** (`git log` muestra el commit inicial con `sin-revisar/`). **No he modificado nada.** ¿Quieres que pase a planificar la limpieza (rebuild + medición de `notas-malo` y/o purga de credenciales del historial)?

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas#  docker image ls
                                                 i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   11a3eb1597e8       25.9kB         9.49kB    U
notas-malo:latest    b14a0a201d4b       1.84GB          475MB
notas-web:latest     b4867db1875c        204MB         49.5MB    U
redis:7-alpine       57e4255cc971       57.8MB         16.7MB    U
```

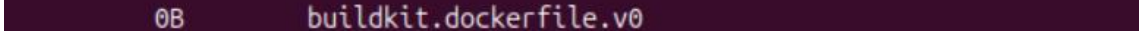
```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker run --rm no
tas-malo id
uid=0(root) gid=0(root) groups=0(root)
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose exe
c web id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker image inspe
ct notas-malo --format '{{.Config.Env}}'
[PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sb
in:/bin PYTHON_VERSION=3.15.1 PYTHON_SHA256=673ae7232d84c894729882c8257d30
0ef2d26523911bfd3917ec8ce7f526f99b REDIS_URL=redis://cache:6379 CLAVE_ADMI
N=admin1234]
```


root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker logs malo-t
est
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deplo
yment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8000
* Running on http://172.17.0.2:8000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 115-172-119
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -sI localhost
:8001/ | grep -i server
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.1.9 Python/3.15.1
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker stop malo-t
est
curl -sI localhost:8000/ | grep -i server
malo-test
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -sI localhost
:8000/ | grep -i server
Server: gunicorn

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker history --n
o-trunc notas-malo | grep -i clave
<missing>
10 minutes ago   ENV CLAVE_ADMIN=admin1234
```




```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker image inspe
ct notas-malo --format 'Cmd={{.Config.Cmd}} Entrypoint={{.Config.Entrypoin
t}}'
Cmd=[/bin/sh -c python app.py] Entrypoint=[]
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker logs malo-t
est
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deplo
yment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8000
 * Running on http://172.17.0.2:8000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 115-172-119
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -sI localhost
:8001/ | grep -i server
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.1.9 Python/3.15.1
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker stop malo-t
est
curl -sI localhost:8000/ | grep -i server
malo-test
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# curl -sI localhost
:8000/ | grep -i server
Server: gunicorn
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker run --rm no
tas-malo ls -a /app
.
..
app.py
requirements.txt
templates
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# cat -n sin-revisar
/Dockerfile sin-revisar/docker-compose.yml
     1  FROM python:latest
     2
     3  RUN apt-get update && apt-get install -y python3-pip curl vim git
     4
     5  WORKDIR /app
     6  COPY . .
     7
     8  RUN pip install -r requirements.txt
     9
    10  ENV REDIS_URL=redis://cache:6379
    11  ENV CLAVE_ADMIN=admin1234
    12
    13  CMD python app.py
    14  services:
    15    web:
    16      build: .
    17      ports:
    18        - "8000:8000"
    19      environment:
    20        - CLAVE_ADMIN=admin1234
    21      depends_on:
    22        - cache
    23      restart: always
    24
    25    cache:
    26      image: redis:latest
    27      ports:
    28        - "6379:6379"
```

Problema señalado	¿Cierto? (evidencia)	Gravedad	Corrección
1. `ENV CLAVE_ADMIN=admin1234` en el Dockerfile (línea 11)	Sí. `docker image inspect notas-malo --format '{{.Config.Env}}'` muestra `CLAVE_ADMIN=admin1234` y `docker history --no-trunc notas-malo | grep -i clave` muestra la capa `ENV CLAVE_ADMIN=admin1234`.	Alta	Quitar el `ENV` de la imagen; si hiciera falta, inyectarla al ejecutar desde un `.env` fuera de git.
2. Servidor de desarrollo con `debug=True` (línea 13)	Sí, `Cmd=[/bin/sh -c python app.py]` y `app.py` línea 38 con `debug=True`: arranca el servidor de desarrollo de Flask (Werkzeug). La versión buena arranca gunicorn . Lo exagerado es decir "ejecución remota de código": el depurador de Werkzeug pide un PIN, aunque `debug=True` nunca debe ir en producción.	Alta	`CMD ["gunicorn", "-b", "0.0.0.0:8000", "-w", "2", "app:app"]`, sin debug.
3. `CMD python app.py` en formato shell (línea 13)	Sí. El build avisa `JSONArgsRecommended` (línea 13) y `Cmd=[/bin/sh -c python app.py]`.	Media	`CMD` en formato JSON (lista).
4. Sin `USER`: corre como root	Sí. `Config.User` vacío y `docker run --rm notas-malo id` → `uid=0(root) gid=0(root)`.	Alta	`USER appuser`.
5. `FROM python:latest` sin fijar (línea 1)	Sí. El build resolvió `python:latest` a Python 3.15.1 (`PYTHON_VERSION` en `Config.Env`), mientras la versión buena fija `python:3.12-slim`. Los builds no son reproducibles.	Media	`python:3.12-slim`.
6. `COPY . .` arrastra `.git`, `sin-revisar` y `.env` (línea 6)	No en este entorno. `docker run --rm notas-malo ls -a /app` no muestra `.git` ni `sin-revisar` porque el `.dockerignore` de la raíz se aplica también al construir con `-f sin-revisar/Dockerfile`.	Media	`.dockerignore` con `.git/`, `.env` y `sin-revisar/`.
7. `ENV REDIS_URL` horneada (línea 10)	Sí. `Config.Env` contiene `REDIS_URL=redis://cache:6379`: obliga a reconstruir para cambiarla y solo funciona si el host se llama `cache`.	Media	Definirla en el `environment:` del compose.
8. `apt-get install python3-pip curl vim git` (línea 3)	Sí. Paso 2/5 del build. `vim`, `git` y `curl` no hacen falta en ejecución, y `python3-pip` es redundante (la imagen base ya trae pip).	Media	Eliminar los paquetes extra.
9. Sin `--no-install-recommends` ni limpieza de `/var/lib/apt/lists` (línea 3)	Sí. La línea 3 del `cat -n` no tiene ninguna de las dos cosas.	Baja	`--no-install-recommends` y `rm -rf /var/lib/apt/lists/*` en el mismo `RUN`.

10. `COPY . .` antes de `pip install` (líneas 6-8)	Sí. Pasos 4/5 (`COPY . .`) y 5/5 (`pip install`) del build: cualquier cambio de código invalida la capa de dependencias.	Media	Copiar `requirements.txt` e instalar primero, y luego copiar el código.
11. `pip install` sin `--no-cache-dir` (línea 8)	Sí en el comando (`RUN pip install -r requirements.txt`).	Baja	`pip install --no-cache-dir`.
12. Sin `HEALTHCHECK`	Sí. El `cat -n` no tiene `HEALTHCHECK` y `docker image inspect` muestra `Healthcheck=<nil>`.	Baja	`HEALTHCHECK` contra `/salud`.
13. Sin `EXPOSE 8000`	Sí, pero irrelevante en la práctica: `EXPOSE` solo documenta el puerto, y el acceso lo da el `ports:` del compose.	Baja	`EXPOSE 8000` (opcional).
14. Redis publicado en el host `6379:6379` sin contraseña	Sí. En el servicio `cache` del compose, el `cat -n` muestra `ports: "6379:6379"` y no hay `requirepass`. Cualquiera con acceso a la red del host puede leer y borrar los datos.	Alta	Quitar `ports`; dejar solo `expose` y la red interna.
15. `CLAVE_ADMIN=admin1234` también en el compose	Sí. En el `environment:` de `web` (`cat -n`): queda en claro en el fichero y en git. Igual que el punto 1, la app no la usa.	Media	Quitarla; si hiciera falta, usar un `.env` fuera de git.
16. `image: redis:latest`	Sí. En el servicio `cache` (`cat -n`). Versión sin fijar: actualizaciones inesperadas.	Media	`redis:7-alpine`.
17. Sin volumen ni `appendonly`	Sí, por ausencia: `cache` no tiene `volumes` ni `command` en el `cat -n`. Al recrear el contenedor se pierden las notas.	Media	Volumen con nombre en `/data` y `--appendonly yes`.
18. `depends_on` simple	Sí. En `web` (`cat -n`) es solo la lista de servicios, sin `condition: web` puede arrancar antes de que Redis esté listo.	Media	`condition: service_healthy`.
19. Sin healthcheck en web ni cache	Sí. No hay ningún `healthcheck` en el `cat -n`.	Media	Healthcheck con python/urllib en `web` y `redis-cli ping` en `cache`.
20. Variables solo en `web`; falta `TITULO_APP`	Irrelevante. `app.py` usa `os.environ.get("TITULO_APP", "Tablón de notas")`, así que tiene valor por defecto y no hace falta definirla.	Baja	Ninguna necesaria.
21. `restart: always`	Parcialmente. Es cierto que el compose usa `always`, pero la diferencia con `unless-stopped` es menor (cómo se comporta tras reiniciar el daemon después de una parada manual).	Baja	`restart: unless-stopped`.

22. Sin hardening (`cap_drop`, `read_only`, límites)	Sí, por ausencia en el `cat -n`, pero es una mejora opcional que la práctica no pide.	Baja	Opcional: `cap_drop: [ALL]`, `read_only: true`.
Tamaño de la imagen: ≈550-700 MB y "2,5-3 veces más" (estimación del agente, sin medir)	Sí, y es mayor. `docker image ls`: `notas-malo` 1,84 GB (475 MB de contenido) frente a `notas-web` 204 MB (49,5 MB): unas 9 veces más. La estimación del agente era demasiado baja.	Alta	Todas las correcciones anteriores (base slim, sin paquetes extra, sin `.git`, `--no-cache-dir`).

6) **Seguridad y tamaño (1,5p).** Con la versión buena, comprueba y documenta: el usuario del contenedor, que el `.env` y `.git` **no** están dentro de la imagen, el estado `healthy` y las capas más pesadas.

```
docker compose exec web id
docker compose exec web ls -a /app
docker image ls
docker history notas-web:latest | head
docker image inspect notas-malo --format '{{.Config.Env}}'
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose exe
c web id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
```

El contenedor web corre como appuser (uid 1000) y no como root, gracias al USER appuser del Dockerfile. En notas-malo corría como root.

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose exe
c web ls -a /app
.  ..  __pycache__  app.py  requirements.txt  templates
```

En app, solo hay código de l aapp. .git y .env no están dentro de la imagen porque el .dockerignore los excluye. En esta carpeta no hay un .env real, así que ls comprobaciojn sirve sobre todo para .git.

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker compose ps
NAME            IMAGE           COMMAND                  SERVICE   CREATE
D        STATUS                  PORTS
notas-cache-1   redis:7-alpine  "docker-entrypoint.s…"   cache     33 hou
rs ago   Up 33 hours (healthy)  6379/tcp
notas-web-1     notas-web       "gunicorn -b 0.0.0.0…"   web       33 hou
rs ago   Up 33 hours (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/t
cp
```

Web y cache están en healthy: el healthy de web comprueba salud con Python y el de cache usa redis-cli ping.

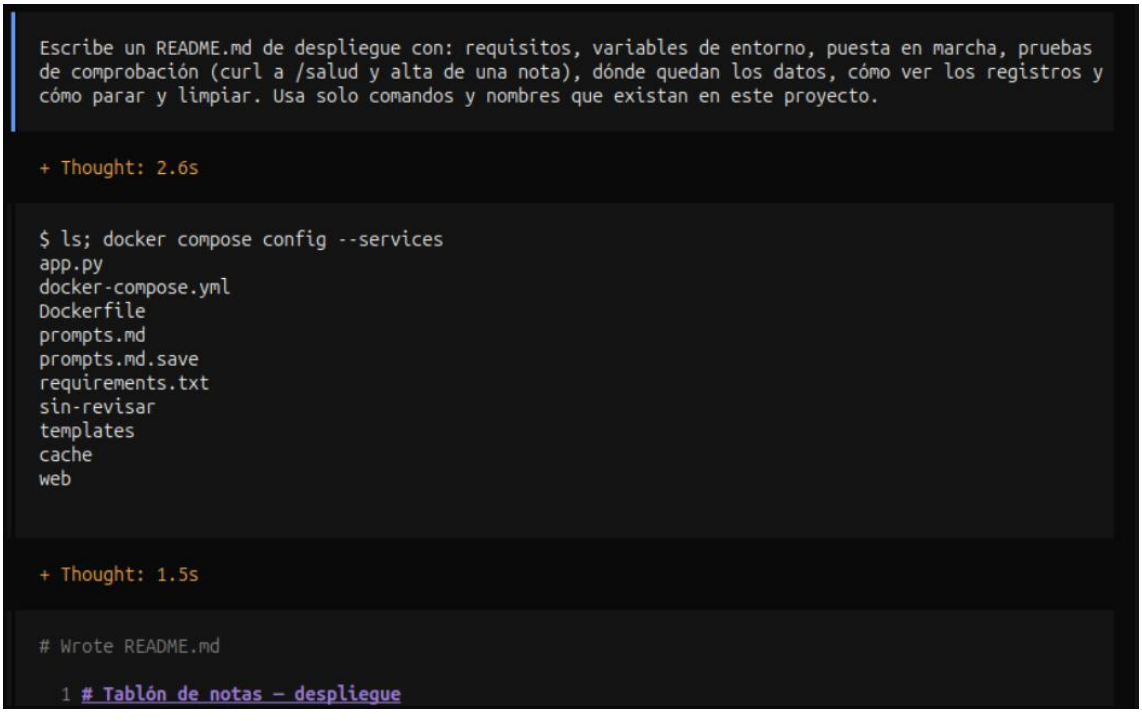
```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker image ls
                                                i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   11a3eb1597e8       25.9kB         9.49kB    U
notas-malo:latest    b14a0a201d4b       1.84GB          475MB
notas-web:latest     b4867db1875c        204MB         49.5MB    U
redis:7-alpine       57e4255cc971       57.8MB         16.7MB    U
```

La version buena pesa unas 9 veces mas porque usa Python:3.12-slim, no instala paquetes extra, usa –no-cache-dir y el .dockerignore deja fuera .git y compañía.

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# docker history not
as-web:latest | head
IMAGE          CREATED        CREATED BY
    SIZE      COMMENT
b4867db1875c   34 hours ago   CMD ["gunicorn" "-b" "0.0.0.0:8000" "-w" "2"
…   0B        buildkit.dockerfile.v0
<missing>      34 hours ago   HEALTHCHECK {Test:[CMD-SHELL python -c "impo
…   0B        buildkit.dockerfile.v0
<missing>      34 hours ago   EXPOSE [8000/tcp]
    0B        buildkit.dockerfile.v0
<missing>      34 hours ago   USER appuser
    0B        buildkit.dockerfile.v0
<missing>      34 hours ago   RUN /bin/sh -c useradd --create-home --shell
…   90.1kB    buildkit.dockerfile.v0
<missing>      34 hours ago   COPY . . # buildkit
    20.5kB    buildkit.dockerfile.v0
<missing>      34 hours ago   RUN /bin/sh -c pip install --no-cache-dir -r
…   20MB      buildkit.dockerfile.v0
<missing>      34 hours ago   COPY requirements.txt . # buildkit
    12.3kB    buildkit.dockerfile.v0
<missing>      34 hours ago   WORKDIR /app
    8.19kB    buildkit.dockerfile.v0
```

La capa mas pesada muestra head, es la de pip install, con 20 MB. El código de la app apenas pesa(20,5 kB).

7) **Documentación del despliegue.** Con ayuda del agente, escribe `README.md` con: requisitos, variables de entorno, puesta en marcha, pruebas de comprobación, dónde quedan los datos, cómo ver los registros y cómo parar y limpiar. Después **bórralo todo** (`docker compose down -v`, `docker image rm`) y repite el despliegue siguiendo solo tu README, para demostrar que es reproducible. (0,5p)



```
alumno@alumno-VMware-Virtual-Platform:~$ cd notas
alumno@alumno-VMware-Virtual-Platform:~/notas$ ls
app.py              Dockerfile  prompts.md.save  requirements.txt  templates
docker-compose.yml  prompts.md  README.md        sin-revisar
alumno@alumno-VMware-Virtual-Platform:~/notas$ cat README.md
# Tablón de notas — despliegue

Aplicación Flask (`app.py`) que guarda notas en una lista de Redis (`notas`, máx
imo 50 entradas) y se sirve con Gunicorn en el puerto **8000**.

## Requisitos

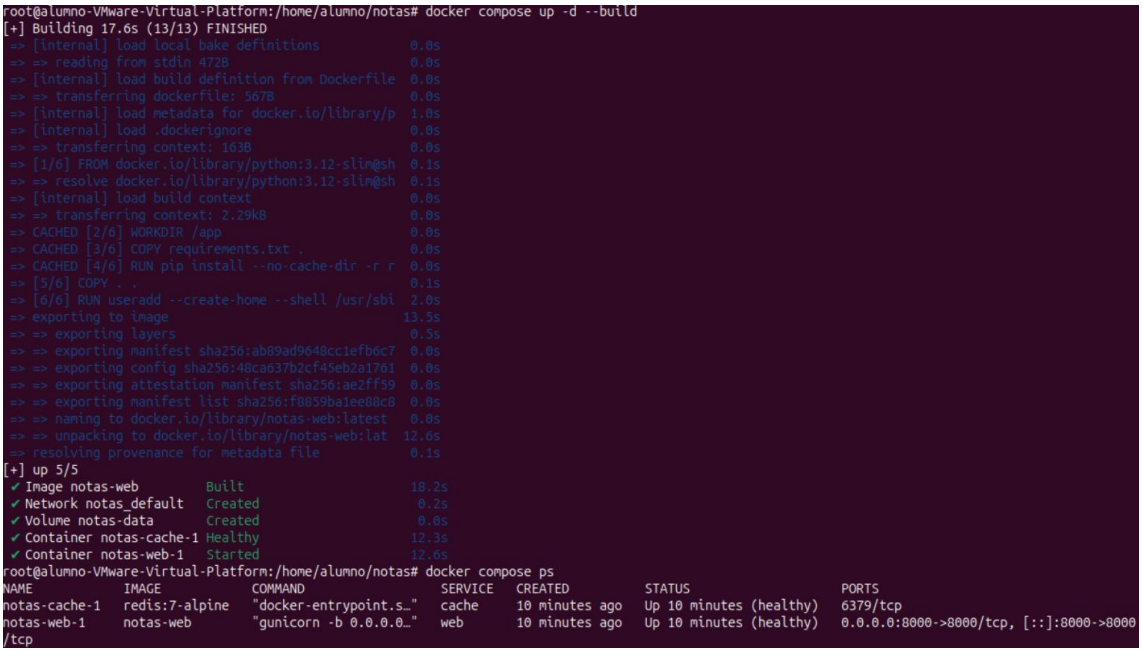
- Docker con el plugin **Compose** (`docker compose version`)

No hace falta instalar Python ni Redis: la imagen los incluye.

## Variables de entorno

| Variable | Obligatoria | Valor por defecto | Descripción |
|---|---|---|---|
| `REDIS_URL` | Sí en contenedor | `redis://localhost:6379` | URL de Redis. El `
```

```
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# doc
ker compose down -v
[+] down 4/4
 ✔ Container notas-web-1   Removed                          4.2s
 ✔ Container notas-cache-1 Removed                          0.7s
 ✔ Volume notas-data       Removed                          0.0s
 ✔ Network notas_default   Removed                          0.3s
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# doc
ker image rm notas-web notas-malo
Untagged: notas-web:latest
Deleted: sha256:b4867db1875c4178d68925c0d852ecf04bf67497421
2a2ff27e4d334af1db424
Untagged: notas-malo:latest
Deleted: sha256:b14a0a201d4b89312b968f33047811f6b2c6fa4bac8
888608bbcf56b229bdf21
root@alumno-VMware-Virtual-Platform:/home/alumno/notas# doc
ker image ls
                                          i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE
hello-world:latest
                     11a3eb1597e8       25.9kB         9.49kB
redis:7-alpine       57e4255cc971       57.8MB         16.7MB
```





Tras borrar los contenedores, volúmenes e imágenes,el despliegue se produce desde cero siguiendo solo con el README, y la app responde con /salud y guarda notas.

Reto (ampliación, no puntúa)

Pide al agente una versión **multietapa** del Dockerfile que instale las dependencias en una etapa y copie solo lo necesario a la final, y compara tamaños. Añade después un servicio `proxy` con Nginx delante de gunicorn y comprueba con `curl -I` qué servidor responde en cada caso.

Registro de prompts

Mantén durante toda la práctica el fichero `prompts.md` con una tabla como esta (una fila por prompt importante):

Nº	Prompt (resumido)	Qué hizo el agente	Qué corregí yo
1	Explica qué hace este proyecto y qué ficheros lo componen	Resumen correcto; se inventó una carpeta tests/	Le indiqué que no existe y lo comprobé con ls

Entrega

Entrega por el aula virtual un PDF con las capturas y las respuestas de cada ejercicio (debajo de cada uno), el fichero `prompts.md` y el enlace al repositorio. Una captura de la respuesta del agente **sin la comprobación** correspondiente no puntúa.